
Entwicklung einer Android-App zur Akkuvorhersage mit TinyML

Portfolio-Dokumentation

Vertiefung I: Mobile und Ubiquitäre Anwendungen

Vorgelegt von: Keno Schürger
Matrikelnummer: 5023033
Studiengang: Wirtschaftsinformatik (BWI)
Dozenten: Prof. Dr. Karsten Huffstadt,
Prof. Dr. Isabel John
Semester: Sommersemester 2026
Bearbeitungszeit: 25. März – 11. Mai 2026

Würzburg, 12. Mai 2026

Inhaltsverzeichnis

Abbildungsverzeichnis	III
Tabellenverzeichnis	III
1 Ausgangssituation und Motivation	1
1.1 Problemstellung	1
1.2 Zielsetzung	1
1.3 Rahmenbedingungen	2
2 Architekturmodell	3
2.1 Gesamtarchitektur im Überblick	3
2.2 Android-App im Detail	5
2.3 Python-Pipeline im Detail	5
3 Datenmodell	6
3.1 CSV-Schema	6
3.2 Warum genau diese zehn Features?	6
3.3 Speicherung und Export	7
3.4 Die App in Aktion	7
4 Programmierung	9
4.1 Datensammlung im Service	9
4.2 Auslesen der 10 Features	10
4.3 On-Device-Inferenz mit TFLite	11
4.4 Robustheit – die App muss überleben	12
4.5 Schema-Drift im Logger	13
4.6 Python-Pipeline – Beispiel Segment-Erkennung	14
4.7 Trainings-Verlauf	14
5 Eigenleistung	16
5.1 Was selbst geschrieben wurde	16
5.2 Was übernommen wurde	16
5.3 Probleme und wie sie gelöst wurden	17
5.4 Ergebnis-Visualisierung	17
6 Datenschutz	18
6.1 Was wird erhoben?	18
6.2 Wo wird gespeichert?	18
6.3 Export	18
6.4 Berechtigungen	19

6.5	Beim Verteilen an Mitstudierende	19
7	KI-Nutzung	20
7.1	Wofür Claude genutzt wurde	20
7.2	Was ich selbst gemacht habe	20
7.3	Bilanz	21
8	Wöchentlicher Erfahrungsbericht	22
8.1	Aufwand	23
9	Kurzreferat-Folien	24
10	Zusammenfassung	24
10.1	Was funktioniert (Stand 11. Mai 2026)	24
10.2	Was noch nicht ideal ist	25
10.3	Was ich aus dem Projekt mitnehme	25

Abbildungsverzeichnis

1	Vereinfachter Datenfluss zwischen den Systemkomponenten	3
2	Detaillierte Gesamtarchitektur des Battery-Predictor-Systems	4
3	Main Activity der Battery Data Collector App auf dem Xiaomi 2107113SG (Redmi Note 10 Pro). Links: erste Sekunden nach Service-Start. Rechts: dasselbe Gerät eine Minute später mit fertiger On-Device-Vorhersage. . . .	8
4	Trainings- und Validation-MAE des Conv1D-Modells. Auf dem Multi-Device-Datensatz erreicht das Modell nach 21 Epochen ein Validation-MAE von ca. 8 h. Da EarlyStopping mit <code>restore_best_weights=True</code> konfiguriert ist, wird das beste Modell zurückgegeben.	15
5	Kumulative Fehlerverteilung der sechs verglichenen Methoden auf dem leakage-freien Test-Set ($n=2,827$). Die x-Achse ist die Fehlertoleranz in Stunden, die y-Achse der Anteil der Vorhersagen innerhalb dieser Toleranz.	17

Tabellenverzeichnis

1	Module der Android-App	5
2	Felder eines CSV-Eintrags	6
3	Code-Umfang nach Bereich	16
4	Im Manifest deklarierte Permissions	19
5	Zeitaufwand nach Woche	23

Listings

1	Mess-Schleife im DataCollectorService	9
2	CPU-Last als Frequenz-Proxy	10
3	App-Kategorisierung (Auszug)	10
4	Hotspot-Status via Reflection	11
5	Inferenz mit TFLite	11
6	Auto-Restart bei onTaskRemoved	13
7	Schema-Drift im Logger abfangen	13
8	Discharge-Segmente finden	14

1 Ausgangssituation und Motivation

1.1 Problemstellung

Smartphone-Akkuanzeigen funktionieren auf den meisten Geräten seit Android 5 (2014) nach einem einfachen Prinzip: man rechnet den durchschnittlichen Verbrauch der letzten Stunden linear in die Zukunft. Wer schon einmal mit 30 % Akku „noch 4 Stunden“ im Display hatte und dann nach einem YouTube-Video plötzlich bei 15 % mit „noch 1 Stunde“ stand, kennt das Problem.

Mit Android 12 hat Google im Oktober 2021 ein eigenes ML-basiertes Modell ins System eingebaut (`PowerManager.getBatteryDischargePrediction()`). Auf Pixel-Geräten nutzt der Settings-Screen diesen Wert; auf Geräten anderer Hersteller (Xiaomi, Samsung) wird er meist gar nicht angezeigt, obwohl die API verfügbar ist.

1.2 Zielsetzung

Ziel ist eine Android-App, die

- alle 30 Sekunden 10 Sensordaten-Features sammelt (Akku, Helligkeit, App-Kategorie, CPU, Temperatur usw.),
- ein eigenes TinyML-Modell on-device einsetzt,
- parallel die Google-API liest und vergleicht,
- alles in eine CSV schreibt, die am PC ausgewertet wird.

Funktional sollte am Ende stehen:

1. Eine Android-App, die im Hintergrund Daten sammelt und überlebt (Foreground-Service, Auto-Restart nach Boot oder Wegwischen).
2. Ein quantisiertes TFLite-Modell (< 15 KB), das in der App on-device läuft und eine Vorhersage in der Notification anzeigt.
3. Eine Trainings-Pipeline am PC, die die exportierte CSV verarbeitet, das Modell trainiert und es zurück in die App deployt.
4. Eine Auswertung, die zeigt, wie gut das eigene Modell gegen die Google-API und einfache Baselines steht.

Dieses Portfolio dokumentiert den *Bau-Aufwand*, die Architektur und die Programmier-Eigenleistung. Die *wissenschaftliche* Auswertung des Multi-Device-Vergleichs ist Gegenstand der separaten Paper-Einreichung (`paper/main.pdf` im selben Repo).

1.3 Rahmenbedingungen

- **Eigenes Gerät:** Xiaomi 2107113SG (Redmi Note 10 Pro), Android 12, 8 GB RAM.
- **Zusätzliche Test-Geräte** (Mitstudierende): Google Pixel 7 Pro, Pixel 8 Pro, Pixel 9 Pro XL.
- **Entwicklungsumgebung:** Android Studio (Kotlin), Python 3.13 mit TensorFlow 2.21 am Windows-Notebook.
- **Bearbeitungszeitraum:** 25. März bis 11. Mai 2026 (sieben Wochen).

2 Architekturmodell

2.1 Gesamtarchitektur im Überblick

Das System gliedert sich in zwei klar getrennte Hälften, die durch CSV-Dateien und das `.tflite`-Modell als Schnittstellen kommunizieren:

1. **Datenerhebung (Android-App)** – alle 30 Sekunden ein Datenpunkt mit 10 Features, gespeichert in CSV.
2. **Modelltraining (Python-Pipeline)** – liest die konsolidierte CSV, trainiert Conv1D, konvertiert nach TFLite.
3. **On-Device-Inferenz (Android-App)** – das TFLite-Modell läuft direkt im Phone und liefert die Vorhersage in der Notification.

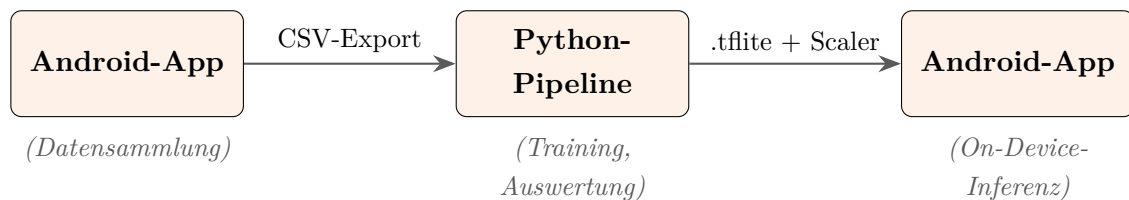


Abbildung 1: Vereinfachter Datenfluss zwischen den Systemkomponenten

Die vollständige Detail-Architektur mit allen Unterkomponenten, internen Verbindungen und Schnittstellen zeigt Abbildung 2 auf der folgenden Seite.

Gesamtarchitektur: Akkuvorhersage mit TinyML

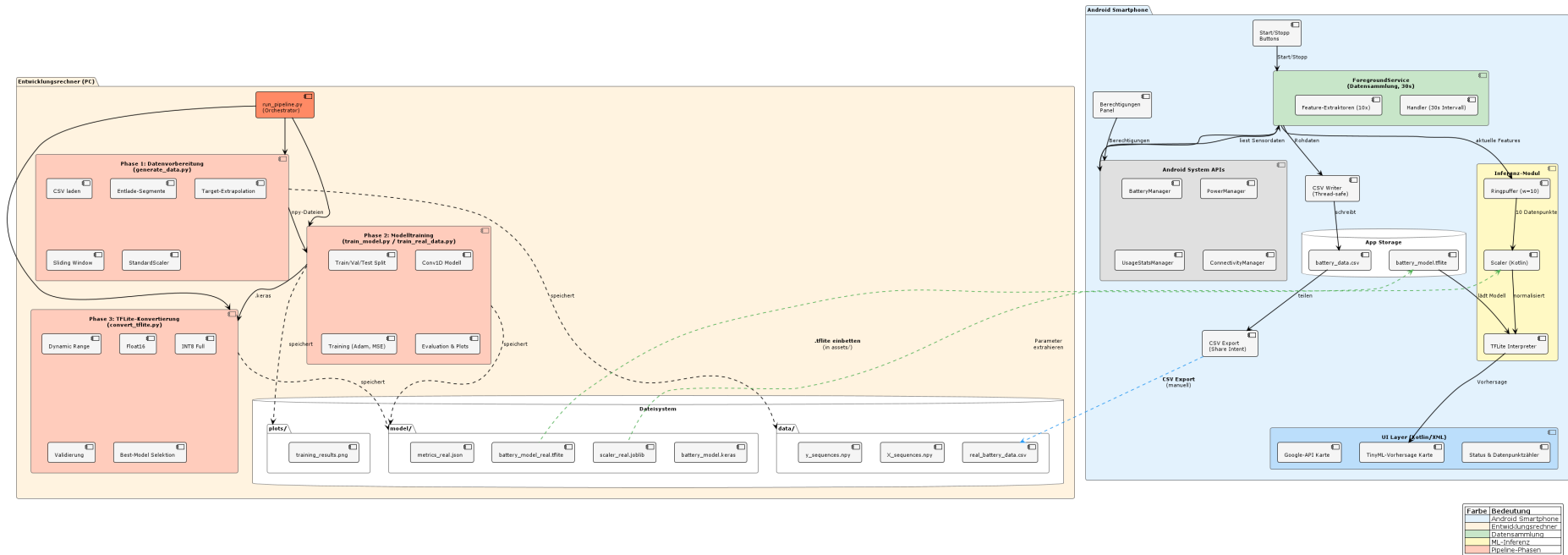


Abbildung 2: Detaillierte Gesamtarchitektur des Battery-Predictor-Systems

2.2 Android-App im Detail

Die App ist in sechs Kotlin-Dateien gegliedert:

Tabelle 1: Module der Android-App

Datei	Aufgabe
MainActivity.kt	UI mit Start/Stop/Export-Buttons, zeigt aktuelle Vorhersage
DataCollectorService.kt	Foreground-Service, sammelt alle 30 Sekunden Daten
BatteryDataPoint.kt	Datenklasse für einen Messpunkt mit CSV-Serialisierung
BatteryDataLogger.kt	Schreibt CSV in filesDir, behandelt Schema-Drift
BatteryPredictor.kt	TFLite-Interpreter mit StandardScaler-Normalisierung
BootReceiver.kt	Startet den Service nach Geräte-Neustart

2.3 Python-Pipeline im Detail

Die Auswerte-Seite ist nach *Forschungs-Methodik* organisiert (eigener Ordner pro Methode, separater Eval-Ordner). Der Orchestrator `run_pipeline.py` bringt sie in Reihenfolge:

1. `methods/tinyml/merge_devices.py` – alle Geräte-CSVs zu einer kombinierten CSV zusammenführen
2. `methods/tinyml/data_prep.py` – Discharge-Segmente finden, Sliding-Windows bilden, Segment-Level-Split
3. `methods/tinyml/train.py` – Conv1D-Modell trainieren
4. `methods/tinyml/tflite_convert.py` – Konvertierung in drei TFLite-Varianten (dynamic, float16, int8)
5. `methods/*/predict.py` – Vorhersagen aller sechs verglichenen Methoden auf dem Test-Set
6. `evaluation/*.py` – diverse Auswertungs-Skripte

3 Datenmodell

3.1 CSV-Schema

Pro Messpunkt schreibt die App eine Zeile mit folgenden Spalten:

Tabelle 2: Felder eines CSV-Eintrags

Spalte	Typ	Bedeutung
session_id	String (8)	Zufalls-ID pro Service-Start
timestamp	long (ms)	Unix-Zeitstempel
datetime	String	YYYY-MM-DD HH:MM:SS
battery_level	Float 0–100	Akku-Stand in Prozent
screen_on	Int 0/1	Bildschirm an?
brightness	Float 0–100	Helligkeit normalisiert
active_app_category	Int 0–5	0=idle, 1=social, 2=video, 3=gaming, 4=browser, 5=productivity
wifi_on	Int 0/1	WiFi-verbunden?
mobile_data_on	Int 0/1	Mobilfunk aktiv?
charging	Int 0/1	Lädt gerade?
cpu_usage	Float 0–100	CPU-Frequenz-Auslastung (Proxy)
temperature	Float (°C)	Akku-Temperatur
hotspot_on	Int 0/1	WLAN-Hotspot aktiv?
system_estimate_min	Float	Google-API-Schätzung in Minuten
system_personalized	Int –1/0/1	Ist Google-Schätzung gerätspezifisch?
own_prediction_h	Float	Eigene TinyML-Vorhersage (Stunden)
linear_baseline_h	Float	Naive Vorhersage aus BatteryManager-Countern

Die ersten drei Spalten sind Metadaten, die zehn nächsten sind die *Features* für das ML-Modell, die letzten vier sind Vergleichswerte zur Auswertung.

3.2 Warum genau diese zehn Features?

Bei der Auswahl gab es eine harte Regel: *ich darf nur lesen, was eine normale App ohne Spezial-Berechtigung lesen kann*. Damit fielen einige naheliegende Kandidaten weg:

- **Genaue Stromstärke pro App** (PowerStats HAL): nur System-Apps.
- **Battery Health** (Verschleißzustand): teilweise gesperrt.
- **Kernel-Fuel-Gauge**: gesperrt.

Was übrig blieb, sind die zehn Features oben. Sie decken drei Gruppen ab: **Akkuzustand**

(level, temperature, charging), **Nutzungsintensität** (screen, brightness, app_category, cpu_usage), **Konnektivität** (wifi, mobile, hotspot).

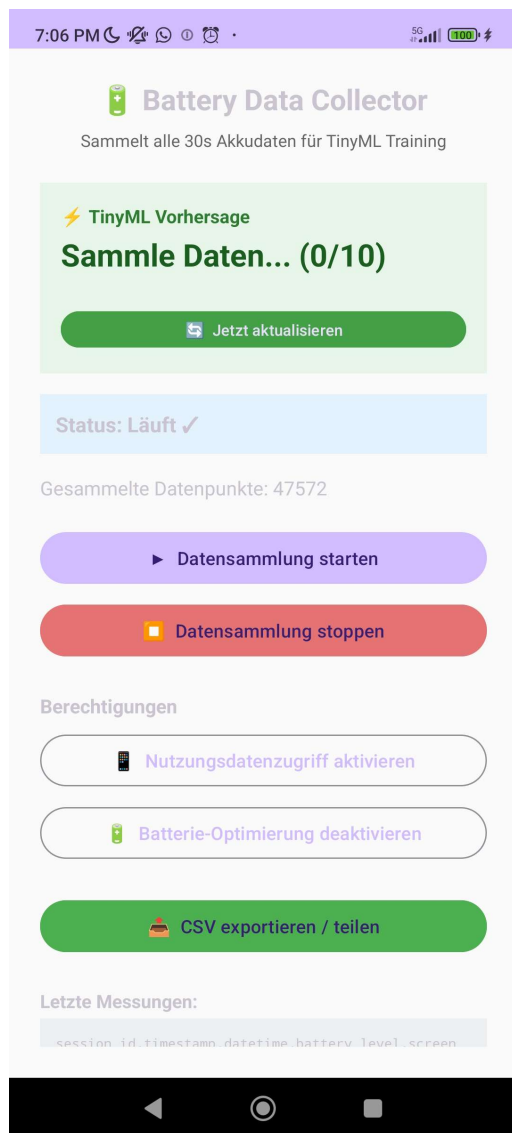
3.3 Speicherung und Export

Die CSV liegt im app-internen Verzeichnis (`Context.filesDir`). Das ist nur für diese App lesbar – weder andere Apps noch ein angeschlossener PC können die Datei direkt sehen (siehe Kapitel 6). Der Export erfolgt explizit per Share-Intent durch den Nutzer.

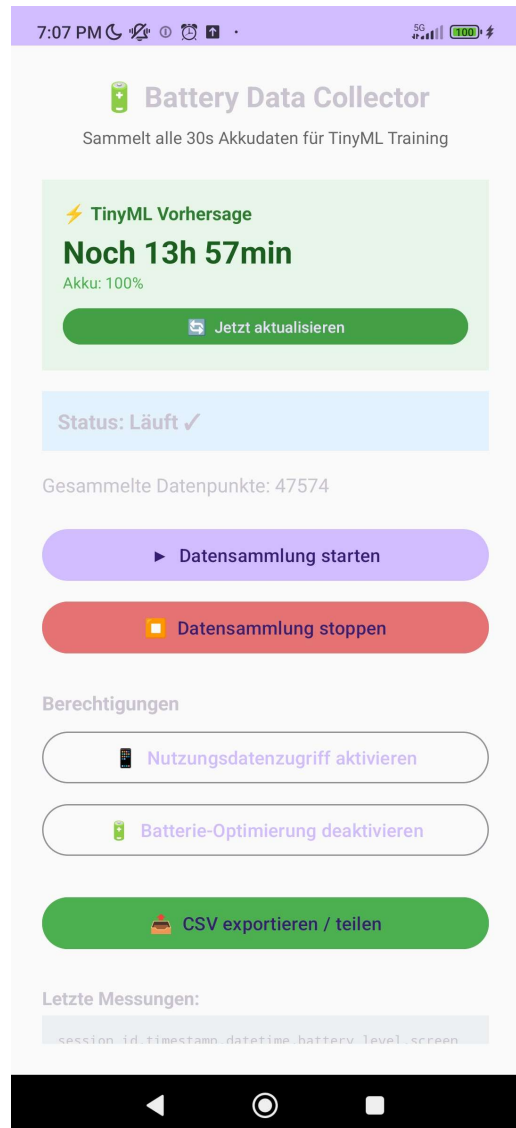
Über vier Geräte hinweg wurden in 45 Tagen **66.001 Messpunkte** in **180 Sessions** gesammelt.

3.4 Die App in Aktion

Abbildung 3 zeigt die Main Activity der App auf dem Xiaomi 2107113SG zu zwei Zeitpunkten: einmal direkt nach dem Service-Start (Modell sammelt erst zehn Datenpunkte, also fünf Minuten Aufwärmphase) und einmal mit aktiver TinyML-Vorhersage. Der Datenpunkt-Zähler im unteren Bereich (**Gesammelte Datenpunkte: 47 572**) ist ein wichtiges UX-Element: er gibt dem Nutzer Feedback, dass der Service tatsächlich noch läuft, ohne dass er die Notification öffnen muss.



(a) Aufwärmphase: „Sammle Daten... (0/10)“. Bis zehn Messpunkte vorliegen, gibt das Modell noch keine Vorhersage aus.



(b) Aktive Vorhersage: „Noch 13 h 57 min“ bei 100 % Akku. Die TinyML-Inferenz läuft direkt auf dem Phone.

Abbildung 3: Main Activity der Battery Data Collector App auf dem Xiaomi 2107113SG (Redmi Note 10 Pro). Links: erste Sekunden nach Service-Start. Rechts: dasselbe Gerät eine Minute später mit fertiger On-Device-Vorhersage.

4 Programmierung

4.1 Datensammlung im Service

Der Kern des DataCollectorService ist ein Handler.postDelayed-Loop, der alle 30 Sekunden eine Messung auslöst:

```

1  private val INTERVAL_MS = 30 * 1000L // 30 Sekunden
2
3  private val collectRunnable = object : Runnable {
4      override fun run() {
5          collectAndLog()
6          handler.postDelayed(this, INTERVAL_MS)
7      }
8  }
9
10 private fun collectAndLog() {
11     // 1) Roh-Features lesen
12     val batteryLevel = getBatteryLevel()
13     val screenOn = if (isScreenOn()) 1 else 0
14     val brightness = getScreenBrightness()
15     // ... (10 Features insgesamt)
16
17     // 2) TinyML-Vorhersage (fail-safe)
18     val features = floatArrayOf(batteryLevel, screenOn.toFloat(), /*
19         ... */
20     var prediction = -1f
21     try {
22         predictor?.let { prediction = it.addDataAndPredict(features)
23     } catch (e: Exception) {
24         Log.e("BatteryCollector", "Prediction failed: ${e.message}")
25     }
26
27     // 3) Vergleichswerte einlesen
28     val systemEstimateMin = getSystemBatteryEstimate()
29     val linearBaselineH = getLinearBaselineHours()
30
31     // 4) Datenpunkt schreiben
32     logger.log(BatteryDataPoint(/* ... */), sessionId)

```

Listing 1: Mess-Schleife im DataCollectorService

4.2 Auslesen der 10 Features

Die meisten Werte kommen direkt aus den Android-System-Services. Drei Sachen waren tricky.

CPU-Auslastung

Seit Android 8 ist `/proc/stat` für Apps gesperrt. Statt der „echten“ CPU-Auslastung lese ich die aktuelle CPU-Frequenz pro Kern und vergleiche mit der Maximalfrequenz. Höhere Frequenz = höhere Last:

```
1 private fun getAvgCpuFrequencyPercent(): Float {
2     val numCores = Runtime.getRuntime().availableProcessors()
3     var totalRatio = 0f
4     var readCores = 0
5     for (i in 0 until numCores) {
6         try {
7             val cur = File("/sys/devices/system/cpu/cpu$i/cpufreq/
              scaling_cur_freq")
8                 .readText().trim().toLong()
9             val max = File("/sys/devices/system/cpu/cpu$i/cpufreq/
              cpuinfo_max_freq")
10                .readText().trim().toLong()
11            if (max > 0) {
12                totalRatio += cur.toFloat() / max
13                readCores++
14            }
15        } catch (_: Exception) { }
16    }
17    if (readCores == 0) throw Exception("No CPU freq readable")
18    return (totalRatio / readCores * 100f).coerceIn(0f, 100f)
19 }
```

Listing 2: CPU-Last als Frequenz-Proxy

Aktive App

Mit dem `UsageStatsManager` (Permission `PACKAGE_USAGE_STATS`, die der Nutzer manuell in den Einstellungen aktivieren muss) lese ich die zuletzt benutzte App und ordne sie über ein Package-Prefix-Mapping einer Kategorie zu:

```
1 private val APP_CATEGORIES = mapOf(
2     "com.instagram"           to 1, // social
3     "com.whatsapp"            to 1,
4     "org.telegram"            to 1,
5     "com.google.android.youtube" to 2, // video
6     "com.netflix"             to 2,
```

```

7      "com.spotify"                to 2,
8      "com.supercell"              to 3, // gaming
9      "com.miHoYo"                to 3,
10     "com.google.android.apps.chrome" to 4, // browser
11     "org.mozilla"                to 4,
12     "com.google.android.apps.docs"  to 5, // productivity
13     "com.microsoft"              to 5,
14     // ... (rund 30 Eintraege)
15 )

```

Listing 3: App-Kategorisierung (Auszug)

Unbekannte Packages bekommen Kategorie 0 (idle), werden aber zusätzlich geloggt, damit ich sie später nachmappen kann.

Hotspot-Status

Die offizielle `WifiManager.isWifiApEnabled()`-API ist als `@hide` markiert, also nicht im SDK. Über Reflection ist sie trotzdem erreichbar:

```

1 private fun isHotspotOn(): Boolean {
2     return try {
3         val wm = applicationContext.getSystemService(Context.
4             WIFI_SERVICE) as WifiManager
5         val method = wm.javaClass.getDeclaredMethod("isWifiApEnabled")
6         method.isAccessible = true
7         method.invoke(wm) as Boolean
8     } catch (e: Exception) {
9         false
10    }
11 }

```

Listing 4: Hotspot-Status via Reflection

4.3 On-Device-Inferenz mit TFLite

Der `BatteryPredictor` hält das TFLite-Modell und einen Ringpuffer mit den letzten 10 Datenpunkten. Erst wenn der Puffer voll ist, wird vorhergesagt. Wichtig: die Features müssen mit denselben `StandardScaler`-Werten normalisiert werden, mit denen das Modell trainiert wurde – die Werte sind hardcoded in der App und werden bei jedem Re-Training neu hineingeschrieben.

```

1 class BatteryPredictor(context: Context) {
2     companion object {
3         const val SEQUENCE_LENGTH = 10

```

```

4      const val NUM_FEATURES = 10
5      // Werte aus models/scaler.joblib (Multi-Device-Training)
6      private val SCALER_MEAN = floatArrayOf(
7          68.79f, 0.83f, 8.47f, 1.31f, 0.08f,
8          0.92f, 0.00f, 46.35f, 33.00f, 0.07f
9      )
10     private val SCALER_SCALE = floatArrayOf(
11         29.61f, 0.38f, 13.01f, 1.17f, 0.27f,
12         0.27f, 1.00f, 16.93f, 4.21f, 0.26f
13     )
14 }
15 private val interpreter: Interpreter
16 private val recentData = ArrayDeque<FloatArray>(SEQUENCE_LENGTH)
17
18 fun addDataAndPredict(features: FloatArray): Float {
19     val normalized = FloatArray(NUM_FEATURES) { i ->
20         (features[i] - SCALER_MEAN[i]) / SCALER_SCALE[i]
21     }
22     if (recentData.size >= SEQUENCE_LENGTH) recentData.
23         removeFirst()
24     recentData.addLast(normalized)
25     if (recentData.size < SEQUENCE_LENGTH) return -1f
26
27     val input = ByteBuffer.allocateDirect(SEQUENCE_LENGTH *
28         NUM_FEATURES * 4)
29         .order(ByteOrder.nativeOrder())
30     for (step in recentData) for (v in step) input.putFloat(v)
31     val output = ByteBuffer.allocateDirect(4).order(ByteOrder.
32         nativeOrder())
33     interpreter.run(input, output)
34     output.rewind()
35     return output.float.coerceAtLeast(0f)
36 }

```

Listing 5: Inferenz mit TFLite

4.4 Robustheit – die App muss überleben

Das war der frustrierendste Teil. Android killt Hintergrund-Services aggressiv, gerade auf Xiaomi-Geräten mit MIUI-Aufsatz. Drei Mechanismen sorgen dafür, dass die Datensammlung nicht abreißt:

1. **Foreground-Service** mit dauerhafter Notification.
2. **Auto-Restart nach Wegwischen** der App-Karte. Wenn der Nutzer die App in der

Übersicht wegwischt, registriert sich der Service über `AlarmManager.setExactAndAllowWhileIdle` einen Restart in 1 Sekunde.

3. **Auto-Restart nach Geräte-Neustart** über einen `BootReceiver`, der das `BOOT_COMPLETED`-Intent abfängt – aber nur, wenn der Nutzer den Sammler explizit aktiviert hatte.

```

1  override fun onTaskRemoved(rootIntent: Intent?) {
2      val restartIntent = Intent(this, DataCollectorService::class.java
3          )
4      val pendingIntent = PendingIntent.getService(
5          this, 99, restartIntent,
6          PendingIntent.FLAG_ONE_SHOT or PendingIntent.FLAG_IMMUTABLE
7      )
8      val alarm = getSystemService(Context.ALARM_SERVICE) as
9          AlarmManager
10     alarm.setExactAndAllowWhileIdle(
11         AlarmManager.ELAPSED_REALTIME_WAKEUP,
12         SystemClock.elapsedRealtime() + 1000,
13         pendingIntent
14     )
15     super.onTaskRemoved(rootIntent)
16 }

```

Listing 6: Auto-Restart bei `onTaskRemoved`

4.5 Schema-Drift im Logger

Beim Hinzufügen neuer Spalten (etwa `system_personalized` oder `linear_baseline_h`) hatten bestehende CSV-Dateien weniger Spalten als die neue Version erwartete. Der Logger prüft deshalb beim Start die Header-Zeile und parkt alte CSVs automatisch:

```

1  init {
2      val expectedHeader = "session_id,timestamp,datetime,{
3          BatteryDataPoint.CSV_HEADER}"
4      if (!csvFile.exists() || csvFile.length() == 0L) {
5          csvFile.writeText("$expectedHeader\n")
6      } else {
7          val firstLine = csvFile.useLines { it.firstOrNull() ?: "" }
8          if (firstLine != expectedHeader) {
9              val backup = File(context.filesDir,
10                  "battery_data_legacy_${System.currentTimeMillis()}.
11                  csv")
12              csvFile.copyTo(backup, overwrite = true)
13              csvFile.writeText("$expectedHeader\n")
14          }
15      }
16 }

```

Listing 7: Schema-Drift im Logger abfangen

4.6 Python-Pipeline – Beispiel Segment-Erkennung

Die wichtigste Pipeline-Funktion findet Entlade-*Segmente*: zusammenhängende Phasen, in denen das Phone entladen wird, ohne zwischendurch zu laden. Daraus werden später Sliding-Window-Sequenzen gebildet.

```
1 def find_discharge_segments(df, min_length, max_gap_ms):
2     segments = []
3     for _, session_df in df.groupby("session_id"):
4         session_df = session_df.sort_values("timestamp").reset_index(
5             drop=True)
6         is_discharging = session_df["charging"] == 0
7         block_id = (~is_discharging).cumsum() # neuer Block bei
8             Laden
9         for _, block in session_df[is_discharging].groupby(block_id[
10             is_discharging]):
11             # innerhalb des Blocks bei groesseren Zeitluecken
12             splitten
13             time_diffs = block["timestamp"].diff()
14             gap_id = (time_diffs > max_gap_ms).cumsum()
15             for _, sub in block.groupby(gap_id):
16                 if len(sub) >= min_length:
17                     segments.append(sub.reset_index(drop=True).copy())
18
19     return segments
```

Listing 8: Discharge-Segmente finden

4.7 Trainings-Verlauf

Abbildung 4 zeigt den Trainings- und Validierungs-Verlauf des Conv1D-Modells über die 21 Epochen, nach denen EarlyStopping greift.

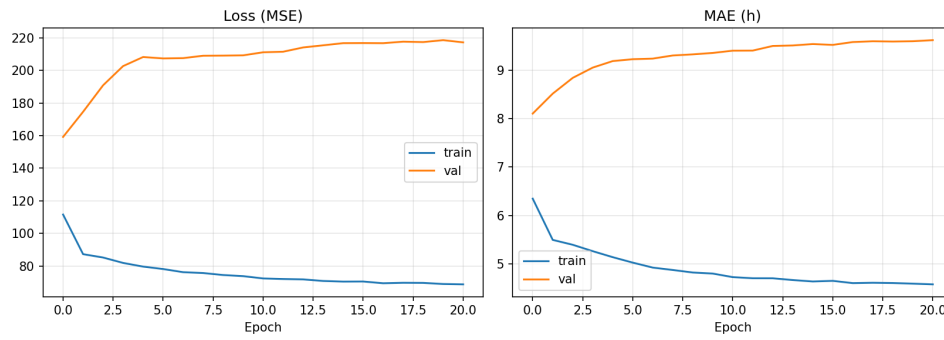


Abbildung 4: Trainings- und Validation-MAE des Conv1D-Modells. Auf dem Multi-Device-Datensatz erreicht das Modell nach 21 Epochen ein Validation-MAE von ca. 8 h. Da EarlyStopping mit `restore_best_weights=True` konfiguriert ist, wird das beste Modell zurückgegeben.

5 Eigenleistung

Im Repo liegen aktuell folgende Code-Mengen:

Tabelle 3: Code-Umfang nach Bereich

Bereich	Zeilen (ohne Leerzeilen)
Android-App (6 Kotlin-Dateien)	≈ 750
Python-Pipeline (<code>methods/</code> , 12 Module)	$\approx 1,050$
Python-Auswertung (<code>evaluation/</code> , 11 Module)	$\approx 1,200$
Konfiguration und Orchestrator	≈ 200
Summe	$\approx 3,200$

5.1 Was selbst geschrieben wurde

- Gesamte Android-App-Struktur (Activity, Service, BroadcastReceiver, FileProvider), inklusive `onTaskRemoved`-Auto-Restart und Schema-Drift-Detection.
- Auswahl und Implementierung der zehn Features (CPU-Frequenz-Proxy, App-Kategorie-Mapping, Hotspot-Reflection).
- Datenmodell, CSV-Schema und FileProvider-Konfiguration für den Share-Export.
- Pipeline-Architektur (sechs verglichene Methoden je eigenes Modul, Trennung von Methoden- und Auswertungs-Schicht).
- Konkrete Implementierung der analytischen Baselines (Linear, Exponential mit Fallback und Cap gegen Divergenz).
- Segment-Level-Split als Anti-Leakage-Mechanismus – der vermutlich wichtigste methodische Beitrag.
- Auswertungs-Skripte: Bootstrap-Konfidenzintervalle, Permutationstests, Per-Device-Analyse, Per-Bucket-Analyse.

5.2 Was übernommen wurde

- Conv1D-Architektur in Anlehnung an gängige TinyML-Topologien (zwei Convolutions, GlobalAveragePooling, zwei Dense).
- TFLite-Konvertierungs-API (Standard-Funktionen von TensorFlow).
- `scikit-learn`: `StandardScaler`, `RandomForestRegressor`, `train_test_split`.

- `scipy: curve_fit` für den exponentiellen Fit.

5.3 Probleme und wie sie gelöst wurden

- *Service stirbt auf MIUI*: Auto-Restart über AlarmManager.
- */proc/stat gesperrt*: Frequenz-Proxy statt echter Auslastung.
- *CSV-Schema-Migration*: Header-Check beim Service-Start.
- *TinyML auf Single-Device zeigt MAE 0.5h Artefakt*: Segment-Level-Split eingeführt, korrigiert den scheinbaren Wert auf 9.96 h.
- *Exponential-Fit divergiert manchmal in 30.000h*: 72-h-Cap als Sanity-Bound.

5.4 Ergebnis-Visualisierung

Abbildung 5 zeigt die kumulative Fehlerverteilung auf dem Test-Set für alle sechs verglichenen Methoden – bei einer Toleranz von 2 h erreichen die Top-Methoden (Linear, Exp, Google-API) rund 60 % Trefferquote.

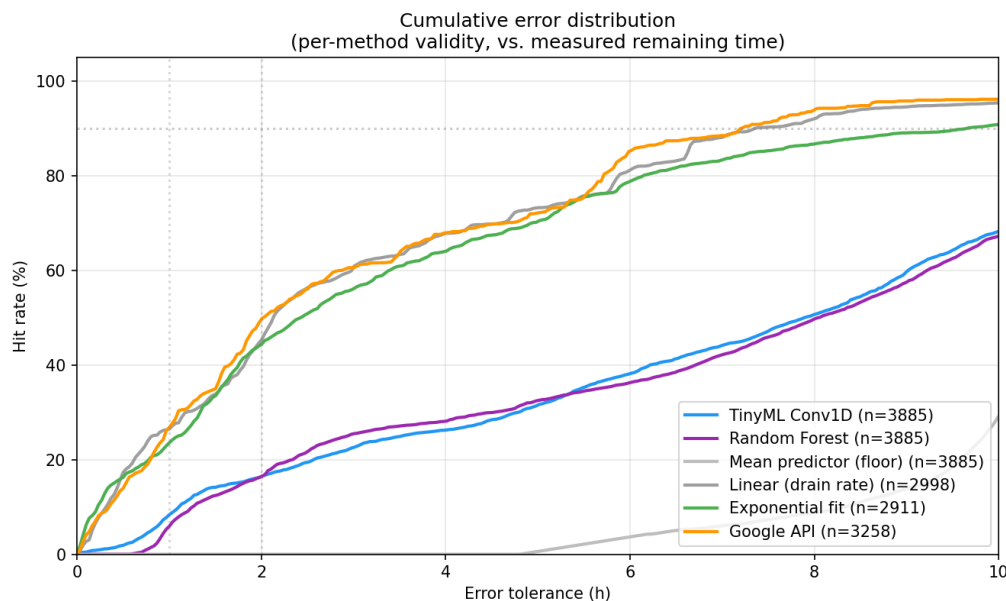


Abbildung 5: Kumulative Fehlerverteilung der sechs verglichenen Methoden auf dem leakage-freien Test-Set ($n=2,827$). Die x-Achse ist die Fehlertoleranz in Stunden, die y-Achse der Anteil der Vorhersagen innerhalb dieser Toleranz.

6 Datenschutz

6.1 Was wird erhoben?

Die App speichert ausschließlich technische Geräte-Metriken (siehe Datenmodell). Es gibt *keinen* Zugriff auf:

- Standort (keine `ACCESS_*_LOCATION`-Permission)
- Kontakte, Anrufprotokolle, SMS
- Mikrofon, Kamera
- Inhalte von Apps (kein Accessibility-Service, kein Screen-Recording)
- Personenbezogene Identifikatoren wie IMEI oder Telefonnummer

Das einzige potenziell sensible Feld ist `active_app_category`, und auch das nur als *Kategorie* (0–5), nicht als konkreter Paketname.

6.2 Wo wird gespeichert?

Im internen Verzeichnis der App (`Context.filesDir`):

- Nur diese App selbst hat Zugriff.
- Andere Apps können nichts lesen (Android-Sandbox).
- Bei Deinstallation werden die Daten automatisch mit entfernt.
- Wird nicht in iCloud / Google Backup synchronisiert.

6.3 Export

Nur über expliziten Nutzerklick (Share-Intent). Es gibt *keinen* automatischen Upload, keinen Server, keine Telemetrie. Der Nutzer entscheidet, ob die CSV per Mail, Drive oder USB-Kabel weitergegeben wird.

6.4 Berechtigungen

Tabelle 4: Im Manifest deklarierte Permissions

Permission	Wofür
FOREGROUND_SERVICE	Background-Service mit Notification
FOREGROUND_SERVICE_SPECIAL_USE	Service-Typ ab Android 14
WAKE_LOCK	Service läuft auch bei Display aus
RECEIVE_BOOT_COMPLETED	Auto-Restart nach Geräte-Neustart
ACCESS_NETWORK_STATE	WiFi-/Mobile-Daten-Status lesen
ACCESS_WIFI_STATE	WiFi-Status, Hotspot-Reflection
PACKAGE_USAGE_STATS	Aktive App lesen (Nutzer muss manuell aktivieren)
POST_NOTIFICATIONS	Vorhersage in Notification anzeigen
REQUEST_IGNORE_BATTERY_OPTIMIZATIONS	Service vor Doze schützen

6.5 Beim Verteilen an Mitstudierende

Für die drei Pixel-Geräte habe ich die App-APK per E-Mail verschickt. Vor der Installation wurde den drei Personen erklärt:

- welche Daten gesammelt werden (CSV mit 14 Spalten),
- wo sie lokal liegen (nur auf ihrem Phone),
- dass die CSV nur zu mir kommt, wenn sie sie aktiv per Share-Intent verschicken,
- dass keinerlei Auto-Upload oder Hintergrund-Übertragung stattfindet.

Eine schriftliche Einwilligung hat jeder per WhatsApp bestätigt.

7 KI-Nutzung

Während des Projekts habe ich **Claude Code** (Anthropic, Modell Opus 4.7 mit 1M-Token-Kontext) als Programmier-Assistent verwendet. Dieser Abschnitt ist bewusst transparent gehalten.

7.1 Wofür Claude genutzt wurde

- **Code-Refactoring:** Umstrukturierung von einem monolithischen Trainings-Skript zu einer modularen Pipeline (`methods/`, `evaluation/`).
- **Boilerplate:** TFLite-Konvertierungs-Boilerplate, Argument-Parser, Standard-Plot-Code (`matplotlib`).
- **Methodische Recherche:** Suche nach relevanter Literatur (Li 2018, Albelali & Ahmed 2025, Flores-Martin 2024, MLPerf Tiny, Harrell 1982).
- **Statistische Tests:** Implementierung des Concordance-Index, Bootstrap-Konfidenzintervalle und Permutationstests nach meinen Vorgaben.
- **Paper-Schreibhilfe:** Strukturierung des LaTeX-Papers, Tabellen-Formatierung, Korrektur-Lesen.
- **Mehrere Audit-Runden:** Die KI hat geholfen, eigene Fehler (falsche Quellen-Autoren, inkonsistente Zahlen, methodische Schwächen wie den Hardware-vs. Datenverteilungs-Confounder) systematisch zu finden.

7.2 Was ich selbst gemacht habe

- Das gesamte Projekt-Konzept und die Forschungsfrage.
- Auswahl der zehn Features (auf welche APIs man als App zugreifen kann).
- Datensammlung auf den vier Geräten – inklusive App-Verteilung an Mitstudierende.
- Alle methodischen Entscheidungen: Segment-Level-Split, C-Index als Metrik, Wahl der Vergleichsmethoden, Bucket-Analyse.
- Kritisches Hinterfragen der KI-Vorschläge, wenn etwas methodisch verdächtig wirkte. Im Verlauf entdeckte die KI auf Nachfrage mehrere eigene Fehler, etwa falsche Autoren bei drei Quellen, erfundene Bucket-Zahlen und zu plakative Aussagen wie „Linear = Google“.
- Alle Anpassungen am Android-Manifest, Permissions, geräte-spezifisches Debugging (MIUI-Service-Stirbt-Problem).

7.3 Bilanz

Realistisch eingeschätzt: ohne KI-Unterstützung hätte ich das Projekt in dieser Tiefe nicht in sieben Wochen geschafft. Die KI war besonders nützlich für *Boilerplate und Recherche-Beschleunigung* sowie für *methodische Disziplin* (Audit-Pass, Konsistenz-Checks über große Code-Mengen).

Was die KI *nicht* ersetzt hat: die Entscheidung, was gemacht wird, das Verstehen der Ergebnisse, die Verteidigung von Trade-offs, und vor allem die handfeste Geräte-Logistik und das Debugging vor Ort.

8 Wöchentlicher Erfahrungsbericht

Woche 1 (KW 13: 23.–29. März)

Setup und erste Datensammlung. Repo angelegt, Android-Projekt aufgesetzt, erste Version des `DataCollectorService` geschrieben. Schon am 25. März lief die erste Messung auf dem Xiaomi durch. Hauptproblem in dieser Woche: die `PACKAGE_USAGE_STATS`-Permission ist nicht via Standard-Dialog vergebbar – der Nutzer muss in den Einstellungen manuell ankreuzen. Ich habe deshalb einen Button in der App eingebaut, der direkt zum `ACTION_USAGE_ACCESS_SETTINGS` springt.

Woche 2 (KW 14: 30. März – 5. April)

Service-Robustheit. Datensammlung bricht beim Wegwischen aus der App-Übersicht ab. Drei Tage damit verbracht herauszufinden, wie man auf Xiaomi-MIUI einen Service vor dem Aggressive-Killing schützt. Lösung: `onTaskRemoved` → `AlarmManager.setExactAndAllowWhileIdle` → Service kommt 1 Sekunde später wieder. Plus `BootReceiver` damit nach Neustart automatisch weitergesammelt wird.

Woche 3 (KW 15: 6.–12. April)

Erstes Modell, erste Auswertung – erstes Erschrecken. Genug Daten zusammen (~11.000 Messpunkte), erstes Training durchgelaufen, MAE 0.53 h. Habe das stolz aufgeschrieben. Dann stutzig geworden: warum ist das so viel besser als die Google-API? Beim Nachprüfen festgestellt, dass das ein klassisches Data-Leakage-Artefakt vom Random-Split war. Pipeline auf Segment-Level-Split umgestellt → MAE wird zu 9.96 h. Das war der methodische Wendepunkt der Arbeit. Am 12. April App-APK an einen Mitstudierenden mit Pixel 7 Pro weitergegeben.

Woche 4 (KW 16: 13.–19. April)

Multi-Device wird real. APK an zwei weitere Pixel-Nutzer verteilt (Pixel 8 Pro, Pixel 9 Pro XL). Schwierig vor allem die Aufklärung darüber, was genau gesammelt wird – ich habe einen Mini-Datenschutz-Hinweis schriftlich verschickt und mündlich erklärt. Alle drei haben zugestimmt. Parallel das CSV-Schema erweitert (`system_personalized`, `linear_baseline_h`) und den Logger so gebaut, dass er bei Schema-Wechsel die alte Datei sicher parkt.

Woche 5 (KW 17: 20.–26. April)

Auswertungs-Tiefe. Pipeline so umgebaut, dass sie alle vier Geräte-CSVs konsolidiert und auswertet. Mean-Predictor und Random Forest als Sanity-Modelle hinzugefügt, weil ein „TinyML schlägt nichts“ wenig aussagekräftig wäre, ohne zu zeigen womit es überhaupt verglichen wird. Dazu noch die analytischen Baselines (Linear, Exponential) implementiert.

Woche 6 (KW 18: 27. April – 3. Mai)

Statistische Härte. Bootstrap-Konfidenzintervalle, Permutationstests, modell-unabhängige Feature-Diagnose (Pearson, Spearman, Mutual Information). Zwischendurch die Xiaomi-Datensammlung beendet (28. April) – die finale Datenmenge stand fest: 66.001 Messpunkte. Erkenntnis dieser Woche: TinyML schlägt auf Multi-Device den Mean-Predictor signifikant ($p \leq 0.005$) – anders als noch auf Single-Device, wo es statistisch indifferent war.

Woche 7 (KW 19: 4.–11. Mai)

Konsolidierung und Paper-Schreiben. Pixel-Datensammlung beendet (10. Mai). Per-Device-Analyse zeigt sehr unterschiedliche Performance pro Gerät – auf Pixel 7 Pro $C=0.79$, auf Xiaomi nur 0.59. Datenverteilungs-Confounder identifiziert: auf Xiaomi war der Akku 88 % der Zeit über 75 %, weshalb wenig Discharge-Dynamik zum Lernen da war. Diese Woche viel Zeit darin investiert, dass das Paper methodisch sauber formuliert ist und die Quellen wirklich das sagen, was zitiert wird (zwei Quellen-Audits, vier korrigierte Autorenangaben, eine korrigierte Größenordnung-Behauptung).

8.1 Aufwand

Tabelle 5: Zeitaufwand nach Woche

Woche	Hauptaufwand	Stunden
1	App-Grundgerüst + Permissions	~ 15
2	Service-Robustheit auf MIUI	~ 18
3	Trainings-Pipeline + Leakage-Aha-Moment	~ 20
4	Schema-Migration + APK-Verteilung	~ 10
5	6-Wege-Vergleich + analytische Baselines	~ 18
6	Statistische Tests + Daten-Diagnose	~ 15
7	Paper schreiben + Audits + Verteidigung	~ 25
Summe		~ 120 h

9 Kurzreferat-Folien

Für die mündliche Verteidigung habe ich eine 16-Folien-Präsentation unter `paper/Verteidigung.pptx` erstellt. Sie deckt ab:

1. Titel und Forschungsfrage
2. Motivation (Settings-App vs. Google-API seit Android 12)
3. Verwandte Arbeiten
4. Datensammlung: 4 Geräte, 45 Tage
5. Sechs verglichene Methoden
6. Methodik-Kernpunkt: Segment-Level-Split
7. Hauptergebnisse (6-Wege-Vergleich)
8. Statistische Signifikanz (Permutationstests)
9. Per-Device-Analyse (Hardware-Effekt)
10. Effizienz-Benchmark (TFLite-Quantisierung)
11. Diskussion: TinyML lernt Signal
12. Diskussion: Google $\approx$ Linear-Baseline
13. Limitations
14. Conclusion (drei Achsen)
15. Q&A

Die Folien sind absichtlich für die wissenschaftliche Paper-Verteidigung optimiert. In diesem Portfolio-Kontext (App-Entwicklungs-Fokus) wären andere Schwerpunkte sinnvoll (Service-Lifecycle, App-Verteilungs-Erfahrung, Geräte-Logistik). Falls ein eigenes Portfolio-Kurzreferat verlangt wird, kann ich daraus eine angepasste Variante extrahieren.

10 Zusammenfassung

10.1 Was funktioniert (Stand 11. Mai 2026)

- Android-App läuft stabil im Hintergrund auf vier verschiedenen Geräten über mehrere Wochen.
- TFLite-Modell ist 14.4 KB groß, läuft auf der CPU des Phones in $\sim 5 \mu\text{s}$ pro Inferenz.

- Datensammlung umfasst 66.001 Messpunkte, sauber per Share-Intent exportiert.
- Trainings- und Auswertungs-Pipeline reproduzierbar via einem Befehl (`python run_pipeline.py`)
- Wissenschaftliches Paper erstellt (separates Dokument).

10.2 Was noch nicht ideal ist

- Die App-UI ist zweckmäßig, aber nicht hübsch. Material-3 ist nur das Default-Theme.
- Die Scaler-Werte werden manuell aus dem Python-Modell ins Kotlin kopiert – automatisierbar via Build-Step wäre eleganter.
- Es gibt kein In-App-Onboarding für die UsageStats-Permission, nur einen Button-Hinweis.

10.3 Was ich aus dem Projekt mitnehme

- **Android im Hintergrund ist hart.** Theoretisch sind Foreground-Services eine Standard-Lösung. Praktisch killt jeder Hersteller anders. Multi-Device-Test ist die einzige Möglichkeit, das wirklich zu prüfen.
- **Methodische Disziplin schlägt Modell-Komplexität.** Der Wechsel von Random-Split zu Segment-Level-Split hatte mehr Einfluss auf das Ergebnis als jedes Hyperparameter-Tuning.
- **Negative Ergebnisse ehrlich darstellen ist genau so wertvoll wie positive.** Anfangs schien das Projekt zu „funktionieren“ (MAE 0.5 h). Erst durch saubere Methodik kam heraus: TinyML funktioniert, aber nicht so spektakulär wie auf den ersten Blick. Das ehrlich aufzuschreiben ist anstrengender, aber wissenschaftlich korrekter.

Repository: <https://github.com/Keno-fsdf/HandyUsageRepo>

Eingereicht am 12. Mai 2026 – Keno Schürger